INGENIERÍA DE MACHINE LEARNING (MLOPS)

Arquitectura de Despliegue y Producción: Modelo Predictivo IVC

Desarrollador: jorber Andrés Pardo Franco

Fase: Producción / Inferencia

Algoritmo Core: LightGBM Regressor

Formato de Serialización: .joblib binario

El presente documento técnico detalla la arquitectura de software y las directrices de ingeniería establecidas para el paso a producción del modelo de estimación del Índice de Vulnerabilidad Climática (IVC) en Bogotá. Superada la fase de experimentación y validación (Jupyter Notebooks / Laboratorio), este protocolo define cómo el conocimiento algorítmico es encapsulado, distribuido y ejecutado en un entorno de servidor para generar predicciones sobre nuevos conjuntos de datos (ej. actualizaciones censales del DANE).

1. El Artefacto de Producción: Serialización del Pipeline

El modelo final no es un script de entrenamiento activo, sino un objeto estático criogenizado mediante el protocolo de serialización `joblib`. Este formato es el estándar de la industria para estructuras de datos en Python basadas en matrices de NumPy (como Scikit-Learn y LightGBM).

> **Contenido del Archivo `modelo_ivc_despliegue_top20.joblib`**
>
> El archivo exportado consolida dos componentes críticos:
> **1. El Preprocesador (Transformer):** Reglas matemáticas estáticas, incluyendo el mapeo de categorías (OneHotEncoder) tal como se observaron en el dataset de entrenamiento original.
> **2. El Estimador (LightGBM):** La arquitectura final de los 100 árboles de decisión por gradiente, con sus nodos y pesos optimizados, listos para recibir los datos transformados y emitir la inferencia.

2. Topología del Flujo de Inferencia

Para garantizar la estabilidad del sistema en producción y evitar fallos por cambios en las estructuras de los datos de entrada, el diseño arquitectónico de inferencia se divide en tres nodos lógicos secuenciales:

Capa de Ingesta Segura	Capa de Reparación y Preprocesamiento	Capa de Inferencia LightGBM
Carga de datos crudos. Filtrado estricto mediante máscara de validación para garantizar **exclusivamente las 20 columnas** requeridas, independientemente del volumen del archivo entrante.	Intervención a nivel de memoria para mitigar bugs de compatibilidad de Scikit-Learn (Passthrough Fix). Aplicación automática de OneHotEncoding a variables nuevas.	Ejecución vectorial de las métricas. Cálculo del IVC (0 a 1). Enrutamiento de las predicciones hacia la base de datos de salida u ordenación de resultados.

3. Estándares de Robustez y Mitigación de Errores (MLOps)

El código de despliegue incorpora estrategias de programación defensiva (*Defensive Programming*) para asegurar alta disponibilidad:

- **Enmascaramiento Restrictivo de Columnas:** En lugar de eliminar identificadores explícitamente (ej. `drop('CD_LC_CM')`), el sistema realiza un *Data Subsetting* estricto: `df_nuevos[top_20_features]`. Esto previene colapsos en la matriz (Shape Errors) si una nueva entrega de datos contiene columnas adicionales no documentadas.
- **Prevención de Data Leakage en Inferencia:** Al cargar el `OneHotEncoder` previamente ajustado, se garantiza que si una categoría nueva o anómala aparece en producción, el parámetro `handle_unknown='ignore'` la descarta de forma segura sin contaminar el cálculo del IVC y sin interrumpir el servicio.
- **Parche de Serialización (Passthrough Fix):** Se implementó un algoritmo dinámico que escanea el objeto `preprocessor` cargado en RAM y sustituye los strings problemáticos de `'passthrough'` (generadores frecuentes de excepciones en diferentes versiones de Python) por un `FunctionTransformer()` vacío, estabilizando el entorno sin alterar la matemática original.

4. Implementación de Referencia en Python

El siguiente bloque ilustra el código de inferencia en producción. Este script es liviano (no requiere entrenamiento) y puede ser ejecutado en entornos serverless (ej. AWS Lambda), microservicios Docker o servidores físicos de la Alcaldía.

```python
# Dependencias del Entorno de Producción
import pandas as pd
import joblib
from sklearn.preprocessing import FunctionTransformer

def fix_passthrough(model):
    """Parche en caliente para compatibilidad de serialización de Scikit-Learn"""
    for i, (name, transformer, cols) in
enumerate(model.named_steps['preprocessor'].transformers):
        if transformer == 'passthrough':
            model.named_steps['preprocessor'].transformers_[i] = (name,
FunctionTransformer(), cols)
    return model

# 1. Carga Segura del Artefacto (El "Cerebro")
path_modelo = 'modelo_ivc_despliegue_top20.joblib'
modelo_despliegue = fix_passthrough(joblib.load(path_modelo))

# 2. Definición del Esquema Estricto (Data Contract)
top_20_features = [
    'TP19_INTE2', 'TP19_EE_E3', 'TP51POSTGR', 'ppt_mean_2000_2018',
    'TP19_EE_E2', 'TP19_EE_E1', 'TP51PRIMAR', 'TP51SUPERI', 'LATITUD',
    'TP14_3_TIP', 'TP34_1_EDA', 'TP19_ACU_2', 'TVIVIENDA', 'LONGITUD',
    'TP19_GAS_1', 'TP19_RECB2', 'TP19_ALC_1', 'AREA', 'TP19_EE_E4', 'TP19_INTE1'
]

# 3. Ingesta de Datos Nuevos (Ej. Actualización DANE)
df_nuevos = pd.read_csv('datos_entrada_produccion.csv')

# 4. Inferencia Vectorizada (Aplicación del Modelo)
df_nuevos['IVC_Predicho'] = modelo_despliegue.predict(df_nuevos[top_20_features])

# 5. Exportación Estratégica (Sectores Prioritarios)
sectores_riesgo = df_nuevos.sort_values(by='IVC_Predicho', ascending=False)
sectores_riesgo.to_csv('reporte_vulnerabilidad_salida.csv', index=False)
```

Fin del Documento Técnico MLOps. Arquitectura validada para integración con APIs de consumo (FastAPI/Flask) y tableros interactivos distritales.